Lab gRPC Service Implementation - Sample Code Guide

Purpose: This guide contains detailed sample code with explanations to help you understand gRPC and Protocol Buffers before attempting the lab tasks.

How to Use This Guide:

1. Read through each sample carefully
2. Run the sample code on your machine
3. Study the explanations to understand how it works
4. Test with provided gRPC client commands
5. Then attempt the corresponding tasks in Lab_Tasks

Table of Contents

1. Part 1: Protocol Buffers Basics
2. Part 2: Simple gRPC Service
3. Part 3: Complete Book Catalog Service
4. Part 4: Error Handling in gRPC
5. Common Mistakes to Avoid
6. Quick Reference

What is gRPC?

gRPC (gRPC Remote Procedure Call) is a modern, high-performance framework developed by Google for building distributed systems and microservices.

Why gRPC?

Traditional REST API:

```
Client → HTTP/JSON → Server
- Text-based (JSON)
- Larger payload size
- Slower parsing
- Manual API documentation
```

gRPC:

```
Client → HTTP/2 + Protobuf → Server
- Binary format (smaller, faster)
- Auto-generated code
- Built-in documentation
- Streaming support
- Type-safe
```

Key Concepts:

1. **Protocol Buffers (protobuf)**: Language for defining data structures
2. **Service Definition**: Define RPC methods in .proto files
3. **Code Generation**: Auto-generate client and server code
4. **Unary RPC**: Single request → single response (like REST)
5. **Streaming RPC**: Multiple messages (we'll cover in later labs)

Part 1: Protocol Buffers Basics

📖 Understanding Protocol Buffers

Protocol Buffers (protobuf) is Google's language-neutral, platform-neutral mechanism for serializing structured data. Think of it as JSON, but faster and smaller!

Create `simple.proto`:

```
syntax = "proto3";

package example;

// Go package option
option go_package = "./proto";

// Simple message type
message Person {
  string name = 1;
  int32 age = 2;
  string email = 3;
  repeated string hobbies = 4;  // Array/slice
}

// Message with nested type
message Address {
  string street = 1;
  string city = 2;
  string country = 3;
  int32 zip_code = 4;
}

message PersonWithAddress {
  Person person = 1;
  Address address = 2;
}

// Enum type
enum BookGenre {
  UNKNOWN = 0;
  FICTION = 1;
  NONFICTION = 2;
  SCIFI = 3;
  FANTASY = 4;
  MYSTERY = 5;
}

// Message using enum
message Book {
  int32 id = 1;
  string title = 2;
  string author = 3;
  BookGenre genre = 4;
  float price = 5;
  bool available = 6;
}
```

📝 Detailed Protobuf Explanation:

Understanding Syntax Declaration:

```
syntax = "proto3";
```

- Declares Protocol Buffers version
- `proto3` is the latest version (simpler than proto2)
- Always put this as the first line

Understanding Package Declaration:

```
package example;
```

- Namespace for your messages (prevents name conflicts)
- Similar to Go packages or Java packages
- Used when importing across multiple .proto files

Understanding Go Package Option:

```
option go_package = "./proto";
```

- Tells protoc where to generate Go code
- `./proto` means generate in `proto/` directory
- Can be full import path like `"github.com/user/project/proto"`

Understanding Message Definition:

```
message Person {
  string name = 1;
  int32 age = 2;
  string email = 3;
}
```

- `message` is like a struct in Go or class in other languages
- Each field has: **type**, **name**, and **field number**
- Field numbers (1, 2, 3) are used in binary encoding
- **Critical:** Never change field numbers (breaks compatibility)

Field Number Rules:

```
1-15:     Use 1 byte  → Use for frequently used fields
16-2047:  Use 2 bytes → Use for less common fields
2048+:    Use 3+ bytes → Rarely used

Reserved: 19000-19999 (protobuf internal use)
```

Understanding Data Types:

Scalar Types:

```
string name = 1;      // Text
int32 age = 2;        // 32-bit integer
int64 id = 3;         // 64-bit integer
float price = 4;      // 32-bit float
double rating = 5;    // 64-bit float
bool available = 6;   // true/false
bytes data = 7;       // Binary data
```

Mapping to Go:

Protobuf Type	Go Type
string	string
int32	int32
int64	int64
float	float32
double	float64
bool	bool
bytes	[]byte

Understanding Repeated Fields (Arrays):

```
message Person {
  repeated string hobbies = 4;
}
```

- `repeated` means this field can have multiple values
- Maps to slice in Go: `[]string`
- Can be empty (length 0)

In Go:

```
person := &Person{
    Name: "John",
    Hobbies: []string{"reading", "gaming", "coding"},
}
```

Understanding Nested Messages:

```
message Address {
  string street = 1;
  string city = 2;
}

message Person {
  string name = 1;
  Address address = 2;  // Nested message
}
```

- Messages can contain other messages
- Like nested structs in Go

In Go:

```
person := &Person{
    Name: "John",
    Address: &Address{
        Street: "123 Main St",
        City: "New York",
    },
}
```

Understanding Enums:

```
enum BookGenre {
  UNKNOWN = 0;      // Default value must be 0
  FICTION = 1;
  NONFICTION = 2;
  SCIFI = 3;
}
```

- `enum` defines a set of named constants
- First value **must** be 0 (default value)
- Values are integers internally

In Go:

```
book := &Book{
    Title: "1984",
    Genre: BookGenre_FICTION,  // Note the prefix
}
```

Compiling Protocol Buffers

Install protoc compiler:

```
# macOS
brew install protobuf

# Ubuntu/Debian
sudo apt install -y protobuf-compiler

# Windows
# Download from: https://github.com/protocolbuffers/protobuf/releases

# Verify installation
protoc --version
```

Install Go protobuf plugins:

```
# Install protoc-gen-go (generates message types)
go install google.golang.org/protobuf/cmd/protoc-gen-go@latest

# Install protoc-gen-go-grpc (generates service code)
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest

# Add to PATH (add to ~/.bashrc or ~/.zshrc)
export PATH="$PATH:$(go env GOPATH)/bin"
```

Compile .proto file:

```
# Create output directory
mkdir -p proto

# Compile
protoc --go_out=. --go_opt=paths=source_relative \
       --go-grpc_out=. --go-grpc_opt=paths=source_relative \
       simple.proto

# This generates:
# - simple.pb.go         (message types)
# - simple_grpc.pb.go    (service interfaces) - only if service defined
```

Understanding the compile command:

- `--go_out=.` → Generate Go code in current directory
- `--go_opt=paths=source_relative` → Keep same directory structure
- `--go-grpc_out=.` → Generate gRPC service code
- `simple.proto` → Input file

Using Generated Code

Create `main.go`:

```
package main

import (
        "fmt"
        "log"

        "google.golang.org/protobuf/proto"
        pb "./proto"  // Import generated code
)

func main() {
        // Create a Person message
        person := &pb.Person{
                Name:    "John Doe",
                Age:     30,
                Email:   "john@example.com",
                Hobbies: []string{"reading", "gaming", "coding"},
        }

        // Print the message
        fmt.Printf("Person: %v\n", person)
        fmt.Printf("Name: %s\n", person.Name)
        fmt.Printf("Age: %d\n", person.Age)
        fmt.Printf("Hobbies: %v\n", person.Hobbies)

        // Serialize to bytes (for network transmission)
        data, err := proto.Marshal(person)
        if err != nil {
                log.Fatal("Marshaling error:", err)
        }
        fmt.Printf("\nSerialized size: %d bytes\n", len(data))

        // Deserialize from bytes
        newPerson := &pb.Person{}
        err = proto.Unmarshal(data, newPerson)
        if err != nil {
                log.Fatal("Unmarshaling error:", err)
        }

        fmt.Printf("\nDeserialized person: %v\n", newPerson)

        // Create nested message
        personWithAddr := &pb.PersonWithAddress{
                Person: person,
                Address: &pb.Address{
                        Street:  "123 Main St",
                        City:    "New York",
                        Country: "USA",
                        ZipCode: 10001,
                },
        }

        fmt.Printf("\nPerson with address: %v\n", personWithAddr)

        // Using enum
        book := &pb.Book{
                Id:        1,
                Title:     "1984",
                Author:    "George Orwell",
                Genre:     pb.BookGenre_FICTION,
                Price:     19.99,
                Available: true,
        }

        fmt.Printf("\nBook: %v\n", book)
        fmt.Printf("Genre: %s\n", book.Genre)
}
```

Run:

```
go mod init proto-example
go get google.golang.org/protobuf/proto
go run main.go
```

Expected Output:

```
Person: name:"John Doe" age:30 email:"john@example.com" hobbies:"reading" hobbies:"
Name: John Doe
Age: 30
Hobbies: [reading gaming coding]

Serialized size: 52 bytes

Deserialized person: name:"John Doe" age:30 email:"john@example.com" hobbies:"readi

Person with address: person:{name:"John Doe" age:30 email:"john@example.com" hobbies

Book: id:1 title:"1984" author:"George Orwell" genre:FICTION price:19.99 available:
Genre: FICTION
```

📝 Key Protobuf Concepts:

1. **Efficient Serialization**: Binary format → smaller than JSON
2. **Schema Evolution**: Can add fields without breaking old code
3. **Language Neutral**: Same .proto file works for Go, Python, Java, etc.
4. **Type Safety**: Compiler checks types
5. **Auto-generated Code**: No manual parsing needed

Part 2: Simple gRPC Service

📖 Sample Code: Hello World gRPC Service

Let's build a simple gRPC service to understand the basics!

Create `greeter.proto`:

```
syntax = "proto3";

package greeter;

option go_package = "./proto";

// Request message
message HelloRequest {
  string name = 1;
}

// Response message
message HelloResponse {
  string message = 1;
  int32 count = 2;
}

// Service definition
service Greeter {
  // Unary RPC: single request → single response
  rpc SayHello(HelloRequest) returns (HelloResponse);

  rpc SayGoodbye(HelloRequest) returns (HelloResponse);
}
```

Compile:

```
mkdir -p proto
protoc --go_out=. --go_opt=paths=source_relative \
       --go-grpc_out=. --go-grpc_opt=paths=source_relative \
       greeter.proto
```

This generates:

- `greeter.pb.go` - Message types
- `greeter_grpc.pb.go` - Service interface and client

Create `server/main.go`:

```
package main

import (
        "context"
        "fmt"
        "log"
        "net"

        pb "../proto"
        "google.golang.org/grpc"
)

// Server implements the Greeter service
type server struct {
        pb.UnimplementedGreeterServer  // Embed for forward compatibility
        callCount int                   // Track number of calls
}

// SayHello implements the SayHello RPC method
func (s *server) SayHello(ctx context.Context, req *pb.HelloRequest) (*pb.HelloRespo
        s.callCount++

        log.Printf("Received SayHello request: name=%s", req.Name)

        // Create response
        message := fmt.Sprintf("Hello, %s! Welcome to gRPC.", req.Name)

        return &pb.HelloResponse{
                Message: message,
                Count:   int32(s.callCount),
        }, nil
}

// SayGoodbye implements the SayGoodbye RPC method
func (s *server) SayGoodbye(ctx context.Context, req *pb.HelloRequest) (*pb.HelloRes
        s.callCount++

        log.Printf("Received SayGoodbye request: name=%s", req.Name)

        message := fmt.Sprintf("Goodbye, %s! Thanks for using gRPC.", req.Name)

        return &pb.HelloResponse{
                Message: message,
                Count:   int32(s.callCount),
        }, nil
}

func main() {
        // Create TCP listener
        lis, err := net.Listen("tcp", ":50051")
        if err != nil {
                log.Fatalf("Failed to listen: %v", err)
        }

        // Create gRPC server
        grpcServer := grpc.NewServer()

        // Register our service
        pb.RegisterGreeterServer(grpcServer, &server{})

        log.Println("🚀 gRPC server listening on :50051")

        // Start serving
        if err := grpcServer.Serve(lis); err != nil {
                log.Fatalf("Failed to serve: %v", err)
        }
}
```

Create `client/main.go`:

```
package main

import (
        "context"
        "fmt"
        "log"
        "time"

        pb "../proto"
        "google.golang.org/grpc"
        "google.golang.org/grpc/credentials/insecure"
)

func main() {
        // Connect to gRPC server
        conn, err := grpc.Dial("localhost:50051",
                grpc.WithTransportCredentials(insecure.NewCredentials()))
        if err != nil {
                log.Fatalf("Failed to connect: %v", err)
        }
        defer conn.Close()

        // Create client
        client := pb.NewGreeterClient(conn)

        // Create context with timeout
        ctx, cancel := context.WithTimeout(context.Background(), time.Second)
        defer cancel()

        // Call SayHello
        fmt.Println("Calling SayHello...")
        helloResp, err := client.SayHello(ctx, &pb.HelloRequest{
                Name: "John",
        })
        if err != nil {
                log.Fatalf("SayHello failed: %v", err)
        }

        fmt.Printf("Response: %s\n", helloResp.Message)
        fmt.Printf("Call count: %d\n\n", helloResp.Count)

        // Call SayGoodbye
        fmt.Println("Calling SayGoodbye...")
        byeResp, err := client.SayGoodbye(ctx, &pb.HelloRequest{
                Name: "John",
        })
        if err != nil {
                log.Fatalf("SayGoodbye failed: %v", err)
        }

        fmt.Printf("Response: %s\n", byeResp.Message)
        fmt.Printf("Call count: %d\n", byeResp.Count)
}
```

Setup and Run:

```
# Initialize module
go mod init grpc-example
go get google.golang.org/grpc
go get google.golang.org/protobuf

# Terminal 1: Start server
cd server
go run main.go

# Terminal 2: Run client
cd client
go run main.go
```

Server Output:

```
🚀 gRPC server listening on :50051
Received SayHello request: name=John
Received SayGoodbye request: name=John
```

Client Output:

```
Calling SayHello...
Response: Hello, John! Welcome to gRPC.
Call count: 1

Calling SayGoodbye...
Response: Goodbye, John! Thanks for using gRPC.
Call count: 2
```

📝 Detailed gRPC Code Explanation:

Understanding Service Definition:

```
service Greeter {
  rpc SayHello(HelloRequest) returns (HelloResponse);
}
```

- `service` defines a set of RPC methods
- Like defining an interface in Go
- `rpc` keyword defines each method
- Format: `rpc MethodName(RequestType) returns (ResponseType);`

Understanding Server Implementation:

```
type server struct {
    pb.UnimplementedGreeterServer  // Embed
}
```

- Must embed `UnimplementedGreeterServer`
- Ensures forward compatibility (if .proto changes)
- Without this, code breaks when adding new methods

```
func (s *server) SayHello(ctx context.Context, req *pb.HelloRequest) (*pb.HelloRespo
    // Implementation
}
```

- First parameter: `context.Context` - for cancellation, timeouts
- Second parameter: pointer to request message
- Returns: pointer to response message + error
- Function signature matches generated interface

Understanding Server Setup:

```
// 1. Create TCP listener
lis, err := net.Listen("tcp", ":50051")
```

```
lis, err := net.Listen("tcp", ":50051")
```

- Opens TCP port 50051
- gRPC uses TCP as transport layer
- Like HTTP servers listening on port 80/443

```
// 2. Create gRPC server
grpcServer := grpc.NewServer()
```

- Creates new gRPC server instance
- Can add interceptors, options here

```
// 3. Register service
pb.RegisterGreeterServer(grpcServer, &server{})
```

- Registers our implementation with gRPC server
- `RegisterGreeterServer` is auto-generated from .proto
- Links service interface to implementation

```
// 4. Start serving
grpcServer.Serve(lis)
```

- Starts accepting connections
- Blocks until server stops
- Handles requests concurrently (goroutines)

Understanding Client Implementation:

```
// 1. Dial server
conn, err := grpc.Dial("localhost:50051",
    grpc.WithTransportCredentials(insecure.NewCredentials()))
defer conn.Close()
```

- `grpc.Dial()` establishes connection to server
- `insecure.NewCredentials()` → no TLS (for development)
- In production, use TLS: `credentials.NewClientTLSFromCert()`
- **Always** close connection with `defer`

```
// 2. Create client
client := pb.NewGreeterClient(conn)
```

- `NewGreeterClient` is auto-generated
- Creates client stub for calling RPC methods
- Like a local object that makes network calls

```
// 3. Create context with timeout
ctx, cancel := context.WithTimeout(context.Background(), time.Second)
defer cancel()
```

- Context controls request lifetime
- `WithTimeout` → request fails after 1 second
- **Always** defer `cancel()` to release resources
- Prevents hanging requests

```
// 4. Call RPC method
response, err := client.SayHello(ctx, &pb.HelloRequest{
    Name: "John",
})
```

- Looks like normal function call!
- Actually sends request over network
- Blocks until response or timeout
- Returns response message + error

Request/Response Flow:

```
Client Side:                    Server Side:
─────────────────────────────────────────────
1. client.SayHello(ctx, req)
2. Serialize request (protobuf)
3. Send over TCP/HTTP2          → 4. Receive request
                                  5. Deserialize
                                  6. Call SayHello()
                                  7. Create response
                                  8. Serialize response
9. Receive response            ← 10. Send over TCP/HTTP2
10. Deserialize
11. Return to caller
```

Key gRPC Concepts:

1. **Unary RPC**: Single request → single response (most common)
2. **Context**: Manages timeouts, cancellation, metadata
3. **Generated Code**: Most code auto-generated from .proto
4. **Transport**: Uses HTTP/2 (multiplexing, streaming)
5. **Binary Protocol**: Faster than JSON

Part 3: Complete Book Catalog Service

📖 Sample Code: Full CRUD gRPC Service

Now let's build a complete service with database!

Create `book_service.proto`:

```
syntax = "proto3";

package bookservice;

option go_package = "./proto";

// Book message
message Book {
  int32 id = 1;
  string title = 2;
  string author = 3;
  string isbn = 4;
  float price = 5;
  int32 stock = 6;
  int32 published_year = 7;
}

// Request messages
message GetBookRequest {
  int32 id = 1;
}

message CreateBookRequest {
  string title = 1;
  string author = 2;
  string isbn = 3;
  float price = 4;
  int32 stock = 5;
  int32 published_year = 6;
}

message UpdateBookRequest {
  int32 id = 1;
  string title = 2;
  string author = 3;
  string isbn = 4;
  float price = 5;
  int32 stock = 6;
  int32 published_year = 7;
}

message DeleteBookRequest {
  int32 id = 1;
}

message ListBooksRequest {
  int32 page = 1;
  int32 page_size = 2;
}

message SearchBooksRequest {
  string query = 1;
}

// Response messages
message GetBookResponse {
  Book book = 1;
}

message CreateBookResponse {
  Book book = 1;
}

message UpdateBookResponse {
  Book book = 1;
}

message DeleteBookResponse {
  bool success = 1;
  string message = 2;
}

message ListBooksResponse {
  repeated Book books = 1;
  int32 total = 2;
  int32 page = 3;
  int32 page_size = 4;
}

message SearchBooksResponse {
  repeated Book books = 1;
  int32 count = 2;
}

// BookCatalog service
service BookCatalog {
  rpc GetBook(GetBookRequest) returns (GetBookResponse);
  rpc CreateBook(CreateBookRequest) returns (CreateBookResponse);
  rpc UpdateBook(UpdateBookRequest) returns (UpdateBookResponse);
  rpc DeleteBook(DeleteBookRequest) returns (DeleteBookResponse);
  rpc ListBooks(ListBooksRequest) returns (ListBooksResponse);
  rpc SearchBooks(SearchBooksRequest) returns (SearchBooksResponse);
}
```

Compile:

```
protoc --go_out=. --go_opt=paths=source_relative \
       --go-grpc_out=. --go-grpc_opt=paths=source_relative \
       book_service.proto
```

Create `server/main.go`:

```
package main

import (
        "context"
        "database/sql"
        "fmt"
        "log"
        "net"
        "strings"

        pb "../proto"
        _ "github.com/mattn/go-sqlite3"
        "google.golang.org/grpc"
        "google.golang.org/grpc/codes"
        "google.golang.org/grpc/status"
)

type bookCatalogServer struct {
        pb.UnimplementedBookCatalogServer
        db *sql.DB
}

func newServer(db *sql.DB) *bookCatalogServer {
        return &bookCatalogServer{db: db}
}

// GetBook retrieves a book by ID
func (s *bookCatalogServer) GetBook(ctx context.Context, req *pb.GetBookRequest) (*p
        log.Printf("GetBook called: id=%d", req.Id)

        var book pb.Book
        err := s.db.QueryRowContext(ctx,
                "SELECT id, title, author, isbn, price, stock, published_year FROM b
                req.Id,
        ).Scan(&book.Id, &book.Title, &book.Author, &book.Isbn, &book.Price, &book.S

        if err == sql.ErrNoRows {
                return nil, status.Errorf(codes.NotFound, "book not found: id=%d", r
        }
        if err != nil {
                return nil, status.Errorf(codes.Internal, "database error: %v", err)
        }

        return &pb.GetBookResponse{Book: &book}, nil
}

// CreateBook adds a new book
func (s *bookCatalogServer) CreateBook(ctx context.Context, req *pb.CreateBookReques
        log.Printf("CreateBook called: title=%s", req.Title)

        // Validate input
        if req.Title == "" || req.Author == "" {
                return nil, status.Error(codes.InvalidArgument, "title and author ar
        }
        if req.Price <= 0 {
                return nil, status.Error(codes.InvalidArgument, "price must be posit
        }

        // Insert into database
        result, err := s.db.ExecContext(ctx,
                "INSERT INTO books (title, author, isbn, price, stock, published_yea
                req.Title, req.Author, req.Isbn, req.Price, req.Stock, req.Published
        )
        if err != nil {
                return nil, status.Errorf(codes.Internal, "failed to create book: %v
        }

        // Get generated ID
        id, _ := result.LastInsertId()

        // Return created book
        book := &pb.Book{
                Id:            int32(id),
                Title:         req.Title,
                Author:        req.Author,
                Isbn:          req.Isbn,
                Price:         req.Price,
                Stock:         req.Stock,
                PublishedYear: req.PublishedYear,
        }

        return &pb.CreateBookResponse{Book: book}, nil
}

// UpdateBook modifies an existing book
func (s *bookCatalogServer) UpdateBook(ctx context.Context, req *pb.UpdateBookReques
        log.Printf("UpdateBook called: id=%d", req.Id)

        // Validate input
        if req.Title == "" || req.Author == "" {
                return nil, status.Error(codes.InvalidArgument, "title and author ar
        }

        // Update in database
        result, err := s.db.ExecContext(ctx,
                "UPDATE books SET title=?, author=?, isbn=?, price=?, stock=?, publi
                req.Title, req.Author, req.Isbn, req.Price, req.Stock, req.Published
        )
        if err != nil {
                return nil, status.Errorf(codes.Internal, "failed to update book: %v
        }

        // Check if book exists
        rowsAffected, _ := result.RowsAffected()
        if rowsAffected == 0 {
                return nil, status.Errorf(codes.NotFound, "book not found: id=%d", r
        }

        // Return updated book
        book := &pb.Book{
                Id:            req.Id,
                Title:         req.Title,
                Author:        req.Author,
                Isbn:          req.Isbn,
                Price:         req.Price,
                Stock:         req.Stock,
                PublishedYear: req.PublishedYear,
        }

        return &pb.UpdateBookResponse{Book: book}, nil
}

// DeleteBook removes a book
func (s *bookCatalogServer) DeleteBook(ctx context.Context, req *pb.DeleteBookReques
        log.Printf("DeleteBook called: id=%d", req.Id)

        result, err := s.db.ExecContext(ctx, "DELETE FROM books WHERE id = ?", req.I
        if err != nil {
                return nil, status.Errorf(codes.Internal, "failed to delete book: %v
        }

        rowsAffected, _ := result.RowsAffected()
        if rowsAffected == 0 {
                return nil, status.Errorf(codes.NotFound, "book not found: id=%d", r
        }

        return &pb.DeleteBookResponse{
                Success: true,
                Message: fmt.Sprintf("Book %d deleted successfully", req.Id),
        }, nil
}

// ListBooks returns paginated books
func (s *bookCatalogServer) ListBooks(ctx context.Context, req *pb.ListBooksRequest
        log.Printf("ListBooks called: page=%d, pageSize=%d", req.Page, req.PageSize)

        // Default pagination
        page := req.Page
        if page < 1 {
                page = 1
        }
        pageSize := req.PageSize
        if pageSize < 1 || pageSize > 100 {
                pageSize = 20
        }

        offset := (page - 1) * pageSize

        // Get total count
        var total int32
        err := s.db.QueryRowContext(ctx, "SELECT COUNT(*) FROM books").Scan(&total)
        if err != nil {
                return nil, status.Errorf(codes.Internal, "failed to count books: %v
        }

        // Get books
        rows, err := s.db.QueryContext(ctx,
                "SELECT id, title, author, isbn, price, stock, published_year FROM b
                pageSize, offset,
        )
        if err != nil {
                return nil, status.Errorf(codes.Internal, "failed to list books: %v"
        }
        defer rows.Close()

        var books []*pb.Book
        for rows.Next() {
                var book pb.Book
                err := rows.Scan(&book.Id, &book.Title, &book.Author, &book.Isbn,
                        &book.Price, &book.Stock, &book.PublishedYear)
                if err != nil {
                        continue
                }
                books = append(books, &book)
        }

        return &pb.ListBooksResponse{
                Books:    books,
                Total:    total,
                Page:     page,
                PageSize: pageSize,
        }, nil
}

// SearchBooks finds books by title or author
func (s *bookCatalogServer) SearchBooks(ctx context.Context, req *pb.SearchBooksRequ
        log.Printf("SearchBooks called: query=%s", req.Query)

        if req.Query == "" {
                return nil, status.Error(codes.InvalidArgument, "search query requir
        }

        searchPattern := "%" + strings.ToLower(req.Query) + "%"

        rows, err := s.db.QueryContext(ctx,
                "SELECT id, title, author, isbn, price, stock, published_year FROM b
                searchPattern, searchPattern,
        )
        if err != nil {
                return nil, status.Errorf(codes.Internal, "search failed: %v", err)
        }
        defer rows.Close()

        var books []*pb.Book
        for rows.Next() {
                var book pb.Book
                err := rows.Scan(&book.Id, &book.Title, &book.Author, &book.Isbn,
                        &book.Price, &book.Stock, &book.PublishedYear)
                if err != nil {
                        continue
                }
                books = append(books, &book)
        }

        return &pb.SearchBooksResponse{
                Books: books,
                Count: int32(len(books)),
        }, nil
}

func initDB() (*sql.DB, error) {
        db, err := sql.Open("sqlite3", "./books.db")
        if err != nil {
                return nil, err
        }

        // Create table
        _, err = db.Exec(`
                CREATE TABLE IF NOT EXISTS books (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        title TEXT NOT NULL,
                        author TEXT NOT NULL,
                        isbn TEXT,
                        price REAL NOT NULL,
                        stock INTEGER DEFAULT 0,
                        published_year INTEGER
                )
        `)
        if err != nil {
                return nil, err
        }

        // Seed data
        var count int
        db.QueryRow("SELECT COUNT(*) FROM books").Scan(&count)
        if count == 0 {
                books := []struct {
                        title, author, isbn string
                        price               float32
                        stock, year         int
                }{
                        {"The Go Programming Language", "Alan Donovan", "978-01341904
                        {"Clean Code", "Robert Martin", "978-0132350884", 42.50, 20,
                        {"Design Patterns", "Gang of Four", "978-0201633610", 54.99,
                }

                for _, b := range books {
                        db.Exec("INSERT INTO books (title, author, isbn, price, stock
                                b.title, b.author, b.isbn, b.price, b.stock, b.year)
                }
                log.Println("✓ Database seeded with sample books")
        }

        return db, nil
}

func main() {
        // Initialize database
        db, err := initDB()
        if err != nil {
                log.Fatalf("Failed to initialize database: %v", err)
        }
        defer db.Close()

        // Create listener
        lis, err := net.Listen("tcp", ":50051")
        if err != nil {
                log.Fatalf("Failed to listen: %v", err)
        }

        // Create gRPC server
        grpcServer := grpc.NewServer()

        // Register service
        pb.RegisterBookCatalogServer(grpcServer, newServer(db))

        log.Println("🚀 BookCatalog gRPC server listening on :50051")

        // Start serving
        if err := grpcServer.Serve(lis); err != nil {
                log.Fatalf("Failed to serve: %v", err)
        }
}
```

Create `client/main.go`:

```
package main

import (
        "context"
        "fmt"
        "log"
        "time"

        pb "../proto"
        "google.golang.org/grpc"
        "google.golang.org/grpc/credentials/insecure"
)

func main() {
        // Connect to server
        conn, err := grpc.Dial("localhost:50051",
                grpc.WithTransportCredentials(insecure.NewCredentials()))
        if err != nil {
                log.Fatalf("Failed to connect: %v", err)
        }
        defer conn.Close()

        client := pb.NewBookCatalogClient(conn)
        ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
        defer cancel()

        // Test 1: List all books
        fmt.Println("=== Test 1: List Books ===")
        listResp, err := client.ListBooks(ctx, &pb.ListBooksRequest{
                Page:     1,
                PageSize: 10,
        })
        if err != nil {
                log.Fatalf("ListBooks failed: %v", err)
        }

        fmt.Printf("Total books: %d\n", listResp.Total)
        for i, book := range listResp.Books {
                fmt.Printf("%d. %s by %s - $%.2f\n", i+1, book.Title, book.Author, b
        }

        // Test 2: Get specific book
        fmt.Println("\n=== Test 2: Get Book ===")
        getResp, err := client.GetBook(ctx, &pb.GetBookRequest{Id: 1})
        if err != nil {
                log.Printf("GetBook failed: %v", err)
        } else {
                book := getResp.Book
                fmt.Printf("Book: %s\n", book.Title)
                fmt.Printf("Author: %s\n", book.Author)
                fmt.Printf("Price: $%.2f\n", book.Price)
                fmt.Printf("Stock: %d\n", book.Stock)
        }

        // Test 3: Create new book
        fmt.Println("\n=== Test 3: Create Book ===")
        createResp, err := client.CreateBook(ctx, &pb.CreateBookRequest{
                Title:         "Learning Go",
                Author:        "Jon Bodner",
                Isbn:          "978-1492077213",
                Price:         44.99,
                Stock:         10,
                PublishedYear: 2021,
        })
        if err != nil {
                log.Printf("CreateBook failed: %v", err)
        } else {
                fmt.Printf("Created book ID: %d\n", createResp.Book.Id)
                fmt.Printf("Title: %s\n", createResp.Book.Title)
        }

        // Test 4: Search books
        fmt.Println("\n=== Test 4: Search Books ===")
        searchResp, err := client.SearchBooks(ctx, &pb.SearchBooksRequest{
                Query: "go",
        })
        if err != nil {
                log.Printf("SearchBooks failed: %v", err)
        } else {
                fmt.Printf("Found %d books:\n", searchResp.Count)
                for _, book := range searchResp.Books {
                        fmt.Printf("- %s by %s\n", book.Title, book.Author)
                }
        }

        // Test 5: Update book
        fmt.Println("\n=== Test 5: Update Book ===")
        updateResp, err := client.UpdateBook(ctx, &pb.UpdateBookRequest{
                Id:            1,
                Title:         "The Go Programming Language (Updated)",
                Author:        "Alan Donovan",
                Isbn:          "978-0134190440",
                Price:         35.99,
                Stock:         25,
                PublishedYear: 2015,
        })
        if err != nil {
                log.Printf("UpdateBook failed: %v", err)
        } else {
                fmt.Printf("Updated book: %s\n", updateResp.Book.Title)
                fmt.Printf("New price: $%.2f\n", updateResp.Book.Price)
        }

        // Test 6: Delete book (commented out to preserve data)
        // fmt.Println("\n=== Test 6: Delete Book ===")
        // deleteResp, err := client.DeleteBook(ctx, &pb.DeleteBookRequest{Id: 4})
        // if err != nil {
        //      log.Printf("DeleteBook failed: %v", err)
        // } else {
        //      fmt.Printf("%s\n", deleteResp.Message)
        // }

        fmt.Println("\n✓ All tests completed!")
}
```

Run:

```
# Terminal 1: Start server
cd server
go run main.go

# Terminal 2: Run client
cd client
go run main.go
```

Expected Output:

```
=== Test 1: List Books ===
Total books: 3
1. The Go Programming Language by Alan Donovan - $39.99
2. Clean Code by Robert Martin - $42.50
3. Design Patterns by Gang of Four - $54.99

=== Test 2: Get Book ===
Book: The Go Programming Language
Author: Alan Donovan
Price: $39.99
Stock: 15

=== Test 3: Create Book ===
Created book ID: 4
Title: Learning Go

=== Test 4: Search Books ===
Found 2 books:
- The Go Programming Language by Alan Donovan
- Learning Go by Jon Bodner

=== Test 5: Update Book ===
Updated book: The Go Programming Language (Updated)
New price: $35.99

✓ All tests completed!
```

📝 Key Implementation Points:

Understanding gRPC Status Codes:

```
return nil, status.Errorf(codes.NotFound, "book not found: id=%d", req.Id)
```

Common gRPC status codes:

Code	Meaning	When to use
OK	Success	(default, don't return explicitly)
InvalidArgument	Bad input	Invalid parameters
NotFound	Resource missing	ID not in database
AlreadyExists	Duplicate	Unique constraint violated
PermissionDenied	Not allowed	Authorization failed
Internal	Server error	Database errors, bugs
Unavailable	Service down	Temporary failures

Understanding Context Usage:

```
func (s *bookCatalogServer) GetBook(ctx context.Context, req *pb.GetBookRequest) (*p
    err := s.db.QueryRowContext(ctx, "SELECT ...", req.Id)
```

- Pass `ctx` to database operations
- Allows cancellation if client disconnects
- Respects timeouts set by client
- **Always** use `Context` versions of DB methods

Understanding Error Handling:

```
if err == sql.ErrNoRows {
    return nil, status.Errorf(codes.NotFound, "book not found")
}
if err != nil {
    return nil, status.Errorf(codes.Internal, "database error: %v", err)
}
```

- Distinguish between "not found" and "error"

- Use appropriate gRPC status codes
- Include helpful error messages
- Don't expose internal details to clients

Part 4: Error Handling in gRPC

📖 Understanding gRPC Errors

Server side - returning errors:

```
// Simple error
return nil, status.Error(codes.InvalidArgument, "title is required")

// Error with formatted message
return nil, status.Errorf(codes.NotFound, "book not found: id=%d", id)

// Error with details (advanced)
st := status.New(codes.InvalidArgument, "invalid book data")
st, _ = st.WithDetails(&pb.BadRequest{
    FieldViolations: []*pb.BadRequest_FieldViolation{
        {Field: "title", Description: "title is required"},
        {Field: "price", Description: "price must be positive"},
    },
})
return nil, st.Err()
```

Client side - handling errors:

```
resp, err := client.GetBook(ctx, &pb.GetBookRequest{Id: 999})
if err != nil {
    // Convert to gRPC status
    st, ok := status.FromError(err)
    if !ok {
        // Not a gRPC error
        log.Printf("Unknown error: %v", err)
        return
    }

    // Check status code
    switch st.Code() {
    case codes.NotFound:
        fmt.Println("Book not found")
    case codes.InvalidArgument:
        fmt.Printf("Invalid input: %s\n", st.Message())
    case codes.Internal:
        fmt.Println("Server error, please try again later")
    default:
        fmt.Printf("Error: %s\n", st.Message())
    }

    return
}

// Use response
fmt.Printf("Book: %s\n", resp.Book.Title)
```

Common Mistakes to Avoid

❌ Mistake 1: Forgetting to Embed Unimplemented Server

```
// Wrong - breaks when .proto changes
type server struct {
    // Missing embed
}
```

✅ Solution: Always Embed

```
type server struct {
    pb.UnimplementedBookCatalogServer  // Always include
    db *sql.DB
}
```

❌ Mistake 2: Not Using Context

```
// Wrong - can't cancel or timeout
err := s.db.QueryRow("SELECT * FROM books WHERE id = ?", req.Id)
```

✅ Solution: Use Context Methods

```
// Correct - respects cancellation
err := s.db.QueryRowContext(ctx, "SELECT * FROM books WHERE id = ?", req.Id)
```

❌ Mistake 3: Wrong Error Codes

```
// Wrong - using generic error
if err == sql.ErrNoRows {
    return nil, fmt.Errorf("not found")
}
```

✅ Solution: Use gRPC Status Codes

```
if err == sql.ErrNoRows {
    return nil, status.Error(codes.NotFound, "book not found")
}
```

❌ Mistake 4: Not Closing Client Connection

```
// Wrong - resource leak
conn, err := grpc.Dial("localhost:50051", options...)
// Forgot to close!
```

✅ Solution: Defer Close

```
conn, err := grpc.Dial("localhost:50051", options...)
defer conn.Close()  // Always close
```

❌ Mistake 5: No Timeout on Client

```
// Wrong - can hang forever
ctx := context.Background()
client.GetBook(ctx, req)
```

✅ Solution: Use Timeout

```
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()
client.GetBook(ctx, req)
```

Quick Reference

Protocol Buffers

```
// Basic message
message Book {
  int32 id = 1;
  string title = 2;
  repeated string tags = 3;  // Array
}

// Enum
enum Status {
  UNKNOWN = 0;  // Must be 0
  ACTIVE = 1;
  INACTIVE = 2;
}

// Service
service BookService {
  rpc GetBook(GetBookRequest) returns (GetBookResponse);
}
```

Compile Proto

```
protoc --go_out=. --go_opt=paths=source_relative \
       --go-grpc_out=. --go-grpc_opt=paths=source_relative \
       file.proto
```

Server Setup

```
// 1. Create server
type server struct {
    pb.UnimplementedServiceServer
}

// 2. Implement methods
func (s *server) Method(ctx context.Context, req *pb.Request) (*pb.Response, error) {
    // Implementation
}

// 3. Start server
lis, _ := net.Listen("tcp", ":50051")
grpcServer := grpc.NewServer()
pb.RegisterServiceServer(grpcServer, &server{})
grpcServer.Serve(lis)
```

Client Usage

```
// 1. Connect
conn, _ := grpc.Dial("localhost:50051",
    grpc.WithTransportCredentials(insecure.NewCredentials()))
defer conn.Close()

// 2. Create client
client := pb.NewServiceClient(conn)

// 3. Call method
ctx, cancel := context.WithTimeout(context.Background(), time.Second)
defer cancel()
resp, err := client.Method(ctx, &pb.Request{})
```

Error Handling

```
// Server
return nil, status.Error(codes.NotFound, "not found")
return nil, status.Errorf(codes.InvalidArgument, "invalid: %s", field)

// Client
st, ok := status.FromError(err)
if ok {
    switch st.Code() {
    case codes.NotFound:
        // Handle not found
    case codes.InvalidArgument:
        // Handle bad input
    }
}
```

---

gRPC vs REST Comparison

Feature	REST	gRPC
Protocol	HTTP/1.1	HTTP/2
Payload	JSON (text)	Protobuf (binary)
Size	Larger	Smaller (50-70% reduction)
Speed	Slower	Faster (10-30x)
Browser Support	Native	Needs proxy
Streaming	Limited	Built-in
Code Generation	Manual or tools	Automatic
Type Safety	Runtime	Compile-time

When to Use Each:

Use REST when:

- Public API for web browsers
- Human-readable data needed
- Simple CRUD operations
- Wide language support needed

Use gRPC when:

- Microservice communication
- Mobile apps (bandwidth limited)
- Real-time features needed
- High performance required
- Strong typing desired

Best Practices

✅ DO:

1. **Always embed Unimplemented server**
2. **Use context for cancellation**
3. **Return proper gRPC status codes**
4. **Validate input before processing**
5. **Use timeouts on client calls**
6. **Close connections with defer**
7. **Log important operations**
8. **Handle errors gracefully**

❌ DON'T:

1. **Don't change proto field numbers**
2. **Don't ignore context**
3. **Don't return generic errors**
4. **Don't trust client input**
5. **Don't forget to close connections**
6. **Don't expose internal errors**
7. **Don't use blocking operations without timeout**

Next Steps

Now that you've studied the sample code:

1. ✅ Make sure you understand Protocol Buffers
2. ✅ Understand service definition
3. ✅ Run all sample servers and clients
4. ✅ Test with different inputs
5. ✅ Experiment with error cases
6. ➡️ **Now attempt the tasks in Lab_Tasks**

Good luck! 🚀

End of Lab Sample Guide